

Mini Project 1: Text Generation with 4-gram Language Models

Natural Language Processing | Master of Data Science
Corpus topic: Khmer Wikipedia articles related to Angkor Wat

1. Introduction

The purpose of this mini project is to build a statistical language model that can learn word-sequence patterns from Khmer text and generate new text from a starting phrase. I selected Angkor Wat as the main topic because it has enough historical, cultural, and architectural vocabulary to make the generation task meaningful. Instead of using only one short article, I combined five related Khmer Wikipedia articles so that the model could learn from a larger and more consistent corpus.

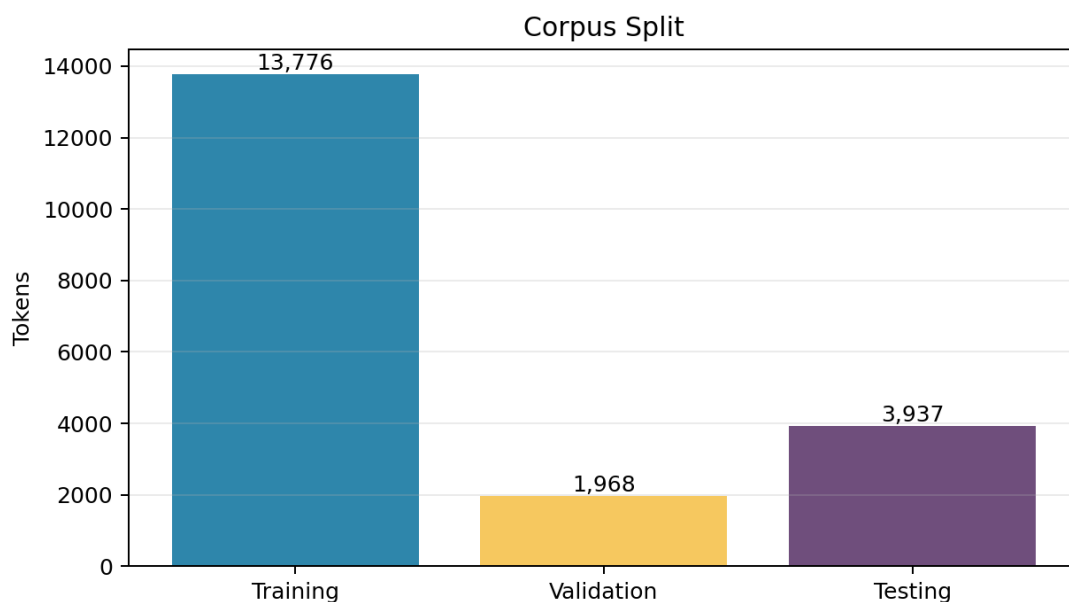
The project compares two required 4-gram language models. The first model is an unsmoothed backoff model, and the second model is an interpolated model with add-k smoothing. Both models are evaluated using perplexity on a separate test set.

2. Corpus Collection and Preprocessing

The corpus was collected from five related Khmer Wikipedia pages: Angkor Wat, Angkor Thom, Bayon Temple, Siem Reap Province, and the Khmer Empire. These pages are related to the same historical and cultural domain, so combining them increases corpus size without mixing unrelated topics.

Corpus statistic	Value
Number of articles	5
Corpus characters	96,199
Tokenizer	khmernltk.word_tokenize
Total tokens	19,681
Training tokens	13,776
Validation tokens	1,968
Testing tokens	3,937
Vocabulary size used	500

During preprocessing, I removed English-only headings and artificial article markers, separated Khmer sentence punctuation, and used khmernltk.word_tokenize to improve Khmer token quality. Tokens outside the limited vocabulary were replaced with <UNK>, which directly follows the project instruction.



The chart above confirms that the dataset follows the required 70/10/20 structure. The training set is used to learn n-gram counts, the validation set is used to tune LM2 hyperparameters, and the test set is kept for final evaluation.

3. Language Model Methods

A 4-gram model predicts the next token using the previous three tokens as context. For example, if the context is 'ပြည်သူ့ အဖွဲ့အစည်း', the model estimates which token is most likely to come next based on patterns counted in the training data.

LM1 uses a backoff strategy. It first tries to use the full 4-gram probability. If that exact 4-gram was not observed in training, it backs off to trigram, then bigram, and finally unigram probability. This approach works well when exact local patterns exist in the corpus.

LM2 uses interpolation. Instead of choosing only one n-gram level, it combines unigram, bigram, trigram, and 4-gram probabilities using lambda weights. Add-k smoothing is also applied so unseen n-grams receive a small non-zero probability.

4. Hyperparameter Tuning

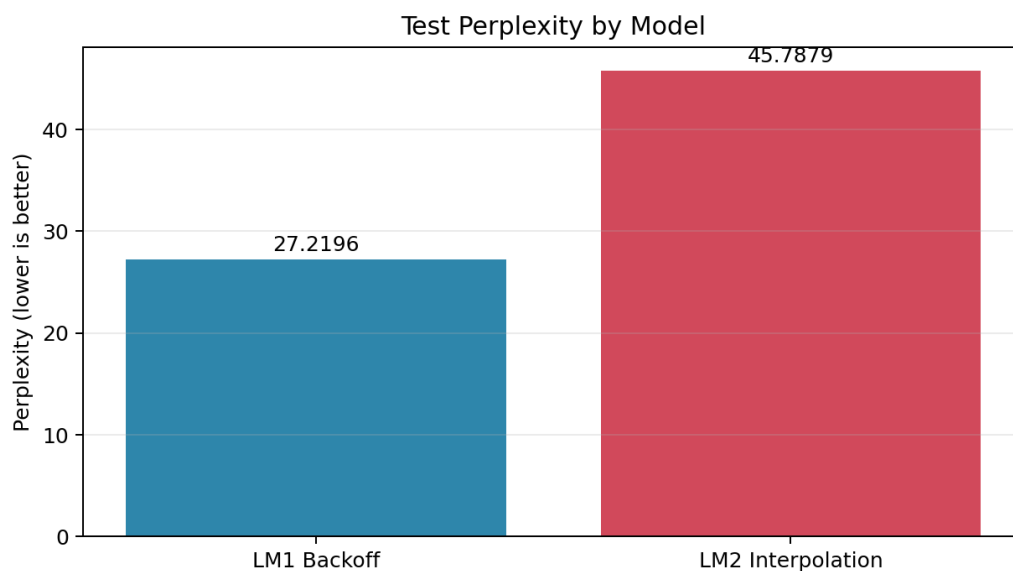
For LM2, I used grid search to tune lambda values and the smoothing value k. Each lambda combination controls how much weight is given to unigram, bigram, trigram, and 4-gram probabilities. The value k controls how much smoothing is applied. The best combination is selected by the lowest validation perplexity.

Hyperparameter	Selected value	Meaning
Lambdas	(0.4, 0.3, 0.2, 0.1)	40% unigram, 30% bigram, 20% trigram, 10% 4-gram
k	0.001	Very small smoothing, suitable for this sparse n-gram setting
Validation perplexity	47.4493	Best LM2 validation score found by grid search

The selected lambda values show that lower-order n-grams are more useful for this corpus. This is reasonable because exact 4-gram patterns are still sparse, even after increasing the corpus size.

5. Main Evaluation Result

Model	Method	Test perplexity
LM1	Unsmoothed backoff 4-gram	27.2196
LM2	Interpolation with add-k smoothing	45.7879

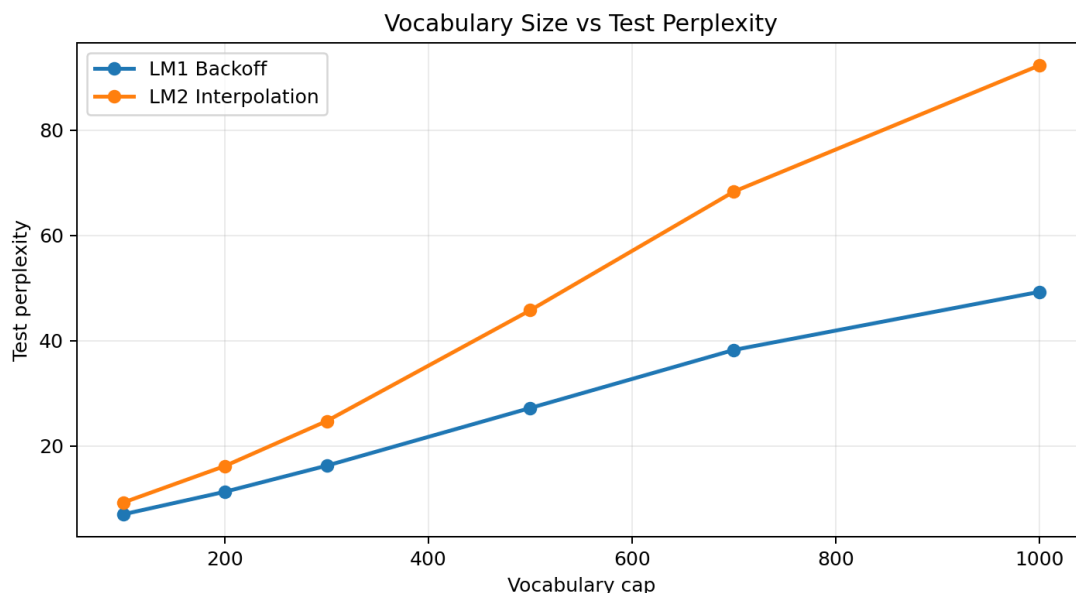


Lower perplexity means the model predicts the test data better. LM1 achieves a test perplexity of 27.2196, while LM2 achieves 45.7879. Therefore, LM1 performs better in this experiment. This does not mean interpolation is always worse; it means that for this Khmer corpus, vocabulary setting, and tokenizer, backoff handled the sparse 4-gram patterns more effectively.

6. Vocabulary Size Experiment

I also tested different vocabulary sizes because vocabulary size has a strong effect on both perplexity and readability. A smaller vocabulary converts more rare words to <UNK>, which can reduce perplexity but makes generated text less readable. A larger vocabulary keeps more real words, but it increases sparsity and makes prediction harder.

Vocab cap	Actual vocab	Test <UNK>	LM1 PP	LM2 PP
100	100	2344	7.0151	9.2468
200	200	2021	11.3045	16.1981
300	300	1783	16.2645	24.74
500	500	1433	27.2196	45.7879
700	700	1233	38.2488	68.3284
1000	1000	1119	49.2772	92.2916



The graph shows that perplexity increases as vocabulary size increases. This happens because the model has more possible next tokens to choose from and many tokens appear only a few times. I selected vocabulary size 500 as a balanced setting: it keeps more real Khmer words than 100 while avoiding the extreme sparsity of 1000.

7. Text Generation

The text generator starts from a seed phrase and predicts the next token repeatedly. For presentation, LM1 is recommended because it has lower test perplexity and usually produces cleaner short output. A length of 20 to 30 tokens is easier to read than a very long generation.

Seed	Model	Generated example
ប្រាសាទ	LM1 Backoff	ប្រាសាទ នេះ កសាង នៅ ឆ្នាំ សតវត្សរ៍ទី ១២ និង ១៣។
ប្រាសាទ	LM2 Interpolation	ប្រាសាទ នេះ បង្ហាញ ឲ្យដឹង ថា គ្រូបង្ហាញ នៅថា <UNK> ទើប នៅក្នុង រូបរាង <UNK> <UNK> របស់ ព្រះអង្គ <UNK>។

The generated text is not perfect, which is expected for an n-gram model. N-gram models do not understand meaning like modern neural language models; they generate text by reusing statistical patterns from the corpus.

8. Conclusion

This project satisfies the required tasks: corpus selection, 70/10/20 data splitting, tokenization, vocabulary limiting with <UNK>, two 4-gram language models, validation-based tuning, perplexity evaluation, and text generation. The strongest result is that LM1 Backoff performs better than LM2 Interpolation on the final test set.

The most important lesson from the experiment is the trade-off between perplexity and readability. A small vocabulary gives lower perplexity but more <UNK> tokens, while a larger vocabulary produces more readable text but increases sparsity and perplexity. The project also shows that Khmer tokenization quality matters, so using a Khmer-specific tokenizer improves the linguistic quality of the input tokens.

References

1. https://km.wikipedia.org/wiki/ប្រាសាទ_អង្គរវត្ត
2. <https://km.wikipedia.org/wiki/អង្គរធំ>
3. <https://km.wikipedia.org/wiki/ប្រាសាទបាយ័ន>
4. <https://km.wikipedia.org/wiki/ខេត្តសៀមរាប>
5. <https://km.wikipedia.org/wiki/ចិត្តភពខ្មែរ>